CapCut Pro — Engenharia Reversa

Oct 13, 2026 · @Kleber

Escopo e alvo

A análise cobriu o CapCut para Windows versão **9.5.0.4050**, instalado em `C:\Users\leona\AppData\Local\CapCut\Apps\9.5.0.4050`. O objetivo foi entender o funcionamento do produto, em especial como o gating "Pro" opera — não reconstruir ou contornar nada.

O pedido original apontava para `C:\caminho\do\app.exe`, que é um placeholder e não existe no disco. O alvo real foi localizado e usado no lugar dele.

Item	Valor
Versão analisada	9.5.0.4050 (de 6 instaladas, da 8.9.1.3802 à 9.5.0.4050)
`EffectPlatform.dll`	SHA-256 `95e79ed511930906a412a50be6c27b058db454e12de61d08d225d49de39d911d`, PE x86-64
Painel H5 analisado	`image_h5_text_effect_publish`, SHA-256 `d52f9470e366c9b21f2ab32c3b91c83377e546884fb15cdb7e4cde44e23c80b6`
Tipo de evidência	Estática: binários, bundles JS, traduções e configs. Sem execução nem captura de rede.

Todas as conclusões abaixo separam o que foi **observado** do que foi **inferido**, e o que permaneceu **desconhecido**.

Arquitetura do app

O CapCut desktop é um híbrido de três camadas de UI sobre um motor de mídia nativo muito grande. Isso explica por que o gating Pro aparece em lugares diferentes: parte é nativa, parte é JavaScript.

Camada	Implementação	Papel
UI nativa	Qt 6 (`Qt6Quick`, `Qt6Widgets`, QML)	Janela principal, timeline, painéis do editor

Camada	Implementação	Papel
UI web embutida	CEF / Chromium (`CefCreator.dll` , `PlatinumWebView.dll` , `cef/`)	Loja, publicação de material, fluxos de conta
UI dinâmica	Lynx, framework da ByteDance (`lynx.dll` 24 MB, `lynx_core.js`)	Painéis remotos atualizáveis sem update do app
Motor de vídeo	`VECreator.dll` **262 MB**, `cccreator.dll` 158 MB, `videoeditor.dll` 71 MB	Composição, render, lógica do produto
IA on-device	`lens.dll` 88 MB, `deepagents_capi.dll` 42 MB, `speechsdk.dll`	Matting, beleza, legendas, TTS
Codecs	FFmpeg (`avcodec-61` , `avformat-61` , `SvtAv1Enc` , `dav1d`)	Decode e encode
Material remoto	`EffectPlatform.dll` , `TTNetDownloaderCrossPlatform.dll` , `res_pool.dll`	Download de efeitos, sons, templates

Dois detalhes de infraestrutura apareceram no JS: o app carrega flags de ambientes internos da ByteDance — `useBOE` / `boeChannel` e `usePPE` / `ppeChannel` — que são os ambientes de teste e pré-produção. Os painéis H5 usam **MobX** para estado e conversam com o app nativo por uma **JSBridge**.

Telas e navegação

O CapCut tem **uma única janela principal** e troca de "cena" dentro dela. Não há pilha de telas nem janelas por seção: `qrc:/SceneManager/main_window/MainWindow.qml` é o único `MainWindow` do app inteiro, e o `SceneManager` substitui o conteúdo.

Onde as telas moram

As 743 telas próprias do app não estão no disco — estão compiladas dentro do `VECreator.dll` como recurso Qt, endereçadas por `qrc:/` . Os 354 arquivos `.qml` soltos na pasta de instalação são **todos** do framework (estilos `QtQuick/Controls` : Basic, Material, Universal, Imagine, Fusion, Windows), nenhum é tela do produto.

Módulo	Telas `.qml`	Papel
`FunctionPanel`	159	Painel de propriedades do item selecionado
`GeneralBiz`	113	Fluxos de negócio (IA, gravação, popups)

Módulo	Telas `.qml`	Papel
`MaterialPanel`	105	Biblioteca de material (mídia, texto, música, stickers)
`HomeScene`	71	Tela inicial, rascunhos, launchpad
`TimeLine`	69	Linha do tempo
`GeneralBizUI`	55	Componentes compartilhados, janelas de mensagem
`Agent`, `Export`, `Cover`, `Player`	21 / 20 / 18 / 17	Copiloto, exportação, capa, preview
`VipBiz`, `Purchase`	13 / 4	Gating e compra (ver seção seguinte)

As quatro cenas

Contagem de referências no `VECreator.dll`: `BizScene` 855, `EditorScene` 715, `HomeScene` 400, `ToolScene` 41. A cena de edição escolhe **um entre sete layouts**, todos em `qrc:/SceneManager/editor_mode/view/`:

- `EditorLayout` — o editor padrão
- `RCEditorLayout` — corte rápido (rough cut)
- `AiShortsEditorLayout` — shorts por IA
- `AiTranslateEditorLayout` — tradução
- `MarketingVideoEditorLayout` — vídeo de marketing
- `CaptionTemplateCreatorEditorLayout` — criação de template de legenda
- `TemplatePublishLayout` — publicação de template

Ou seja: os "modos" do CapCut não são telas separadas, são **layouts alternativos da mesma cena de editor**.

Três sistemas de navegação, um por camada de UI

Camada	Como endereça uma tela	Evidência
Qt nativo	`qrc:/<Módulo>/<pasta>/<Tela>.qml`, troca de cena + 50 janelas de topo (`*Window.qml`) para diálogos	743 caminhos `qrc:` no `VECreator.dll`
Lynx	Tabela de rotas em arquivo: `videofusion://main/lynx?channel=…&bundle=…&window_width=…&window_height=…`	`Resources/lynx_config/lynx_config.json`, 22 rotas

Camada	Como endereça uma tela	Evidência
H5 / CEF	Rotas aninhadas por arquivo, no estilo EdenX (framework da ByteDance)	`nestedRoutes.json`, 14 páginas de publicação

A rota do Lynx **carrega o tamanho da janela**: `window_width` e `window_height` vêm na própria URL, então abrir a tela e dimensioná-la é uma operação só.

E, repetindo o padrão do gating: **a tabela de rotas é dado, não código.** A string `videofusion://` não aparece em nenhum binário do app — nem em ASCII nem em UTF-16 — em `VECreator.dll`, `cccreator.dll`, `FusionUI.dll`, `videoeditor.dll`, `CefCreator.dll`, `lynx.dll` ou `CapCut.exe`. Ela existe apenas no JSON em disco.

O gerenciador de popups fala as três línguas

Em `qrc:/GeneralBiz/common_popup_manager/` há quatro tipos de popup lado a lado: `NativePopupWindow`, `H5PopupWindow`, `LynxPopupWindow` e `NotificationPopupWindow`. É um único sistema de popup capaz de renderizar qualquer uma das três pilhas de UI — o que permite trocar a implementação de uma tela sem mexer em quem a chama.

Deeplinks

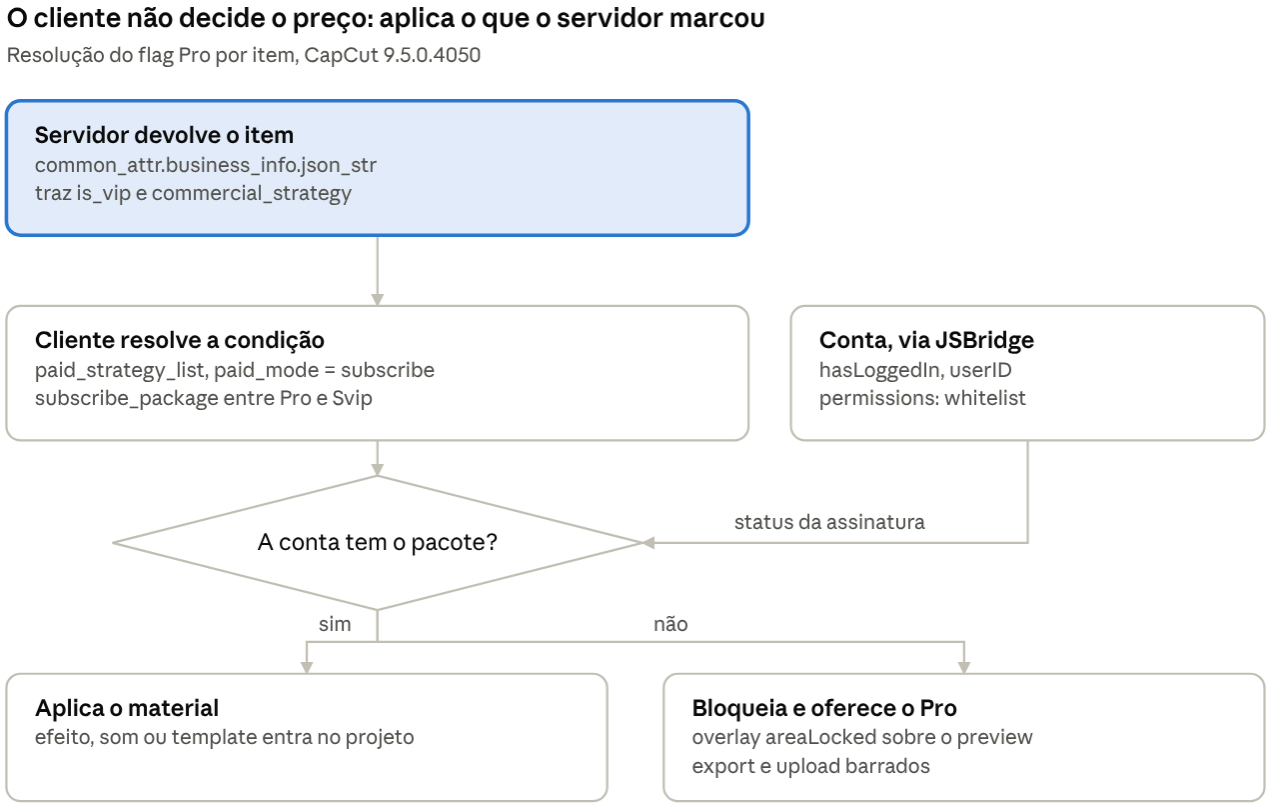
O esquema externo é `capcut://com.ies.videocut/<módulo>/<rota>`. Os pontos de entrada de recurso usam o prefixo `uganchor/anchor_point`, com um parâmetro `featureEntry` para atribuição:

```
capcut://com.ies.videocut/copilot/plugin/ai_shorts
capcut://com.ies.videocut/copilot/plugin/ai_matting
capcut://com.ies.videocut/uganchor/anchor_point/ai_rough_cut?featureEntry=
capcut://com.ies.videocut/uganchor/anchor_point/caption_auto_subtitles?
featureEntry=
capcut://com.ies.videocut/uganchor/anchor_point/video_one_click_beauty?
featureEntry=
```

Um detalhe de transição

A tela inicial existe em duas gerações convivendo no mesmo binário: `HomeScene/Views/HomePage.qml` e `HomePageTabView.qml` (antiga) ao lado de `HomeScene/Launchpad/Views/HomePageX.qml` e `HomePageTabViewX.qml` (nova, sufixo `X`). Reescrita em andamento, provavelmente atrás de flag.

Como funciona o CapCut Pro



fluxo do gating Pro · 4 etapas, 1 decisão

A condição comercial não vive no cliente: ela chega por item, e o cliente apenas a resolve e obedece. Trocar o flag localmente não liberaria nada, porque a exportação e o consumo de créditos são revalidados no servidor.

A lógica de gating, descompilada

Este é o achado central: a decisão "este material é Pro" **não está no código** — ela é calculada em tempo de execução a partir de metadados que o servidor envia por item. A função abaixo foi recuperada de `static/js/publish-text-effect.3a6eb5bb.js`, linha 1, colunas 2284–2288.

O REA marcou o campo como `unknown-coverage` com a justificativa *"Own property is_vip is unknown-coverage. Unknown value at binding"* — prova estática de que o valor vem de fora do binário.

Endpoints do catálogo

- `POST /artist/v1/effect/mget_item` — metadados de itens, em lotes de **50**
- `POST /artist/v1/homepage/get_effect_list` — listagem da vitrine

- `POST /artist/v1/tools/check_antidirt` — moderação de conteúdo

A resolução do flag, deobfuscada

```
const mgetEffectItems = async ({ items, disableRegion, ...rest }) => {
  const out = [];
  for (const chunk of chunked(items, 50)) {
    const resp = await request("/artist/v1/effect/mget_item",
                               { items: chunk, ...rest }, { disableRegion });

    (resp.effect_item_list || []).forEach(item => {
      const { business_info, extra } = item.common_attr;

      if (typeof business_info.json_str === "string") {
        // o flag chega como JSON serializado DENTRO de um campo string
        const { is_vip = false, commercial_strategy } =
              safeJsonParse(business_info.json_str, {});

        let isVip = false;
        if (commercial_strategy === undefined) {
          isVip = is_vip;                       // caminho legado
        } else {
          const { paid_strategy_list = [] } =
                commercial_strategy.resource_export ?? {};
          const sub = paid_strategy_list.find(s => s.paid_mode ===
"subscribe");
          isVip = sub === undefined
            ? is_vip
            : ["Pro", "Svip"].includes(sub.subscribe_package ?? "");
        }

        business_info.json = { is_vip: isVip };  // flag normalizado que a UI
lê
      }

      if (typeof extra === "string")
        item.common_attr.extra_json = safeJsonParse(extra, undefined);
    });

    out.push(resp.effect_item_list);
  }
  return { effect_item_list: out.flat() };
};
```

O que essa função revela

1. **Duas camadas de assinatura existem de fato:** `"Pro"` e `"Svip"`. Isso confirma as famílias de string `vip` e `svip` encontradas nas traduções.
2. **O modelo de cobrança é extensível:** o código filtra `paid_mode === "subscribe"` dentro de uma `paid_strategy_list`, o que implica que outros `paid_mode` existem além de assinatura — compra avulsa ou créditos.
3. **Há compatibilidade retroativa:** sem `commercial_strategy`, cai no booleano `is_vip` antigo.
4. **O flag é normalizado para um único booleano** (`business_info.json.is_vip`) antes de a UI consumir — a UI nunca vê a estratégia comercial crua.
5. **O gating é por item**, não por categoria: cada efeito, som ou template carrega sua própria condição.

Permissões de criador e a ponte com o nativo

Além do gating de consumo, existe um segundo sistema: **capacidades de criador concedidas por whitelist de servidor**. Recuperado de `publish-text-effect.3a6eb5bb.js` e `9807.c566dcfa.js`, coluna 2535.

O app define um enum de permissões nomeadas:

```
{
  StickerPublish:    "sticker_publish",
  StickerOnePackage: "sticker_one_package",
  EffectHighPrio:    "effect_high_prio",
  EffectSetBusiness: "effect_set_business",
  EffectSetVip:      "effect_set_vip",   // permissão de marcar efeito como
VIP
  EffectBatchMore:   "effect_batch_more",
  EffectIconBlank:   "effect_icon_blank"
}
```

E as resolve assim, a partir da conta:

```
async initUserInfoImpl() {
  const acct = await bridgeGetAccount().catch(() => null);   // JSBridge ->
app nativo

  if (acct?.hasLoggedIn === false) {
    this.userInfo = { avatar: "", nickname: "", uid: "" };
    this.permissions = [];
    this.isEnableAuthorization = false;
    return;
```

```
  }

  const [profile, whitelist] = await Promise.all([getProfile(),
getWhitelist()]);

  // permissões = o que o servidor habilitou para esta conta
  this.permissions = whitelist.enable_list
      ?.filter(e => e.is_enable)
      ?.map(e => e.whitelist_type) || [];

  const cfg = await getConfig();
  this.isEnableAuthorization = cfg.allow_pro_template_auto_transition;
}
```

Pontos observados:

- **Nada é decidido localmente.** As permissões vêm de `enable_list`, filtradas por `is_enable` — é um sistema de feature flags por conta.
- **`allow_pro_template_auto_transition`** é um flag de servidor específico de Pro, controlando transições automáticas em templates.
- **A identidade vem do nativo, não do navegador.** `hasLoggedIn`, `userID`, `avatarURL` e `nickname` chegam via JSBridge. O wrapper tem timeout padrão de **3000 ms** e emite o erro `JsbOperationTimeout` com o nome da bridge.
- `EffectSetVip` é permissão **de publicação**, não o gate do consumidor — distinção importante para não confundir os dois sistemas.

Camada nativa: o que deu e o que não deu

`EffectPlatform.dll` — descompilado com sucesso

O Ghidra analisou o DLL inteiro. A busca pelo vocabulário de entitlement (`vip`, `unlock`, `purchase`, `benefit`, `need_pay`, `premium`) retornou **apenas dois símbolos, ambos do runtime C++**:

- `0x180165c7a` — `?_Unlock@?$basic_streambuf@...` (mutex de `std::basic_streambuf`)
- `0x1801660f6` — `_Mtx_unlock`

Zero strings de gating. Resultado negativo, mas conclusivo: este DLL baixa e aplica material, e não decide preço. Converge com o achado do JS — a condição comercial é metadado de servidor.

Onde o gating nativo mora

Triagem por contagem de bytes nos binários do app:

Binário	Ocorrências de `vip`	Leitura
`VECreator.dll`	**1674**	Dono da lógica de UI do gating
`videoeditor.dll`	316	Motor, limites de uso
`VEHelper.exe`, `CefCreator.dll`, `DeepAgentsService.dll`, `VEDetector.exe`	98 cada	Tabela de strings compartilhada, não lógica
`VECrashHandler.exe`	97	Idem

As chaves gettext de UI (`pc_caption_menu_vip`, `pc_super_resolution_vip`, `pc_shareview_vip_upload`) são referenciadas pelo `VECreator.dll`.

O limite duro

`VECreator.dll` tem **262 MB**. O Ghidra precisa concluir a auto-análise do programa inteiro antes de responder qualquer consulta — não há resultado parcial. Duas tentativas:

Tentativa	Timeout	Resultado
1ª	330 s (padrão do REA)	`provider_timeout`
2ª	2.400.000 ms (40 min, via `REA_GHIDRA_STARTUP_TIMEOUT_MS`)	`provider_timeout` após rodar os 40 min completos

A função de gating nativa não foi extraída. Esse é o teto prático: descompilar o monolito do CapCut não é viável com esta toolchain. O entendimento do mecanismo veio da camada JS, que é legível, e convergiu com a evidência nativa disponível.

Efeitos e sons: local vs servidor

A descoberta mais prática desta seção: **o CapCut não empacota catálogo.** O disco traz os *motores* e alguns presets; tudo que o usuário navega como biblioteca é baixado.

Sons — a evidência é contundente

O app inteiro contém **3 arquivos de áudio**:

Arquivo	Função
`Resources/notice_sound/export_success.mp3`	Som de exportação concluída
`Resources/notice_sound/export_failed.mp3`	Som de erro na exportação
`Resources/empty_audio2s.mp3`	Silêncio de 2 s, usado como placeholder

Não existe biblioteca de músicas nem de efeitos sonoros embutida. O que há localmente são **modelos de processamento** de áudio:

- `rnnoise/rnnoise_v2.0.model` — redução de ruído
- `audiosami/unet_denoise_44k_music_model_v1.0.model` — denoise de música
- `audiosami/time_align_*.model` — alinhamento temporal
- `tts/tts_local_auth_code` — apenas credencial de text-to-speech, não vozes

Efeitos — motores locais, catálogo remoto

Presets e motores que vêm no disco, em `Resources/`:

- **Cor e ajuste:** `Chroma` e `Chroma2` (chroma key), `Curves`, `lut/` (LUTs), `LogWheel` e `PrimaryWheel` (color wheels), `MixMode` (modos de mesclagem), `DefaultAdjustBundle`
- **IA on-device** em `Resources/models/`: `facebeautifymodel`, `humanparsingmodel`, `hairparser`, `headsegmodel`, `depthmodel`, `interpolation`, `faceclustingmodel`, `avatar3d`, `invisible_watermark`
- **Remoção de fundo:** `mattingBlend`, `mattingBlendV2`, `matting_config`

O catálogo de efeitos, filtros, stickers, fontes e templates chega via `EffectPlatform.dll` + `TTNetDownloaderCrossPlatform.dll` + `res_pool.dll`, e cada item carrega seu próprio flag `is_vip` conforme a seção anterior.

O overlay de bloqueio

`Resources/areaLocked/AmazingFeature/` contém uma cena de efeito completa (`main.scene`, `content.json`, scripts Lua, `LuaRTTI.MarkGen.lua`). É o overlay visual sobreposto ao preview quando um recurso Pro é usado sem assinatura — ou seja, o bloqueio é renderizado como um efeito, não como um simples aviso.

Créditos de IA: um segundo eixo de cobrança

A assinatura não é o único mecanismo. Recursos de IA generativa consomem **créditos**, contabilizados por operação. Isso significa que ser Pro não dá uso ilimitado de IA — dá uma cota.

Nas traduções pt-BR a mecânica aparece por inteiro:

Chave	Texto	O que revela
`pc_aigc_credit_consume_purchase`	"Aplicar este recurso consumirá %1 créditos."	Custo variável por recurso
`pc_aigc_credit_bar`	"Os créditos de assinatura reiniciam mensalmente"	Cota mensal renovável
`pc_ai_expand_m10n_toast_generated_credits_consumed`	"Imagens geradas. %1 créditos consumidos."	Debitado por geração
`pc_ai_expand_m10n_popup_desc_stop_generate_credits_consumed`	"As imagens não serão geradas. Os créditos ainda serão consumidos."	Não há estorno no cancelamento
`pc_aigc_credit_consume_purchase2`	Instrui comprar créditos no app mobile	Compra avulsa fora do desktop

Duas consequências observadas:

- O sufixo `m10n` (abreviação de *monetization*) nomeia as chaves de IA generativa, confirmando que essa família foi desenhada como produto monetizado à parte.
- Há famílias distintas `pc_aigc_credit_vip_bar` e `pc_aigc_credit_svip_bar`, ou seja **as cotas de crédito diferem entre Pro e Svip** — coerente com `["Pro", "Svip"]` encontrado no código JS.

Toolchain e método

O que foi instalado

Componente	Versão / local
Ghidra	12.1.4 em `C:\Users\leona\Tools\ghidra_12.1.4_PUBLIC`
`GHIDRA_INSTALL_DIR`	Definido de forma persistente via `setx`
Java	JDK 26.0.2 (Ghidra exige 21+)
REA	`rea-agents` 6.4.0, provider `ghidra`

O zip veio do repositório oficial da NSA, `ghidra_12.1.4_PUBLIC_20260921.zip`, 569.649.598 bytes, tamanho verificado após o download.

Evidence IDs

Análise	Evidence ID
Strings do `EffectPlatform.dll`	`ev_aeda6d7e6f6a0d1afb57d7c3c75a7567b2e479139bf0333b990b1110fd7f0314`
Grafo do painel H5	`ev_37c2b2079281ee7a2a9e9949a30757050c9f29204533b57eaa10f5ea14373d82`

O grafo semântico do painel H5 rendeu **11.942 nós** e **4.460 unknowns** sobre 41 fontes parseadas. Os nós relevantes:

- `jsrg_node_a5213f41...` — `destructure:/is_vip`
- `jsrg_unknown_eedd0e72...` — *"Static destructure property /is_vip has unknown-coverage presence"*
- `jsrg_unknown_6ffcc63f...` — *"Static write property /EffectSetVip has unknown-coverage presence"*

Sequência que funcionou

1. Triagem barata primeiro: `grep` de bytes nos binários para localizar qual DLL referencia as chaves de UI. Isso evitou analisar DLLs grandes no escuro.
2. Descompilação nativa dirigida no DLL pequeno e relevante, com `--snapshot` para cachear a análise.

3. Quando o nativo bateu no limite, **pivotar para a camada JS** — que respondeu a pergunta de forma mais direta e legível.
4. Localizar o código pelos offsets exatos (`linha:coluna`) que o grafo semântico reporta, e ler as janelas de bytes com `cut -c` .

A lição de método: em um app híbrido, a camada JavaScript frequentemente carrega a lógica de negócio mais clara, e custa uma fração do esforço de atacar o motor nativo.

Limitações

O que esta análise **não** estabeleceu, declarado sem suavização:

1. **A função de gating nativa não foi lida.** `VECreator.dll` (262 MB) excedeu o Ghidra mesmo com 40 minutos. A lógica nativa de decisão permanece inferida, não observada.
2. **O catálogo real é inacessível localmente.** Não há como enumerar quais efeitos, sons ou fontes são Pro a partir do disco — essa lista vive no servidor, por item.
3. **O JS está minificado e sem source maps.** Os nomes de variáveis no código deste documento foram reconstruídos por leitura; a semântica é fiel, os identificadores locais não são os originais.
4. **Zero observação de runtime.** Nada foi executado, nenhuma requisição foi capturada. Não vi o payload real de entitlement que o servidor devolve, nem como a validação acontece na exportação.
5. **O painel analisado é do criador, não do consumidor.** `image_h5_text_effect_publish` é a UI de publicação de material. A lógica de resolução do `is_vip` é compartilhada e por isso vale, mas o paywall que o usuário final vê é nativo ou carregado remotamente.
6. **Ghidra só enxerga strings definidas.** Literais empacotados ou não definidos podem não aparecer; um resultado vazio não prova ausência.
7. **Uma única versão foi analisada.** Não houve comparação entre as 6 versões instaladas, que revelaria a evolução do modelo comercial — o caminho legado `is_vip` sugere que essa evolução existe.

Como fechar cada lacuna

Lacuna	Caminho
Lógica nativa	IDA Pro (lida melhor com binários enormes) ou análise por seções
Payload de entitlement	Captura de runtime e de rede com o app aberto
Evolução do modelo	`compare_application_versions` entre 8.9.1 e 9.5.0

Lacuna	Caminho
Paywall do consumidor	Observação do CEF em execução

Implicações para construir um editor próprio

A engenharia reversa respondeu uma pergunta de negócio junto com a técnica: **o valor do CapCut Pro não está no código do cliente.** Está no catálogo de servidor, nos modelos de IA treinados e na infraestrutura de CDN. Reconstruir o cliente, mesmo que fosse legalmente possível, não entregaria o produto — e os números desta análise mostram por quê: 262 MB de motor que o descompilador não processa em 40 minutos, mais um catálogo que nem existe no disco.

O caminho produtivo é outro: as *funcionalidades* são técnicas padrão, com implementações abertas maduras. Recursos e ideias não têm copyright — apenas o código específico tem. Construir do zero é legítimo; copiar código descompilado não é.

Recurso gated no CapCut	Equivalente open-source	Observação
Remover fundo	RMBG-2.0, BiRefNet, MODNet (ONNX)	Qualidade atual comparável ou superior
Legendas automáticas	Whisper / faster-whisper + VAD	Remover pausas = VAD + edição de timeline
Super-resolução	Real-ESRGAN	Gated por limite de uso no CapCut
Redução de ruído de áudio	RNNoise	**É exatamente o que o CapCut usa** (`rnnoise_v2.0.model`)
Interpolação de quadros	RIFE	O CapCut traz `interpolation` em `models/`
Chroma key, LUTs, curvas	FFmpeg + shaders GLSL	Técnica clássica
Text-to-speech	Piper, XTTS, Coqui	
Música e efeitos sonoros	Freesound, Pixabay (licenças abertas)	O CapCut também não os possui — licencia

Arquitetura sugerida

Vale copiar do CapCut a **estrutura**, não o código: um núcleo de render separado da UI, e material descrito por metadados (o padrão `business_info.json` é um bom modelo de como descrever um efeito de forma extensível, sem o campo de cobrança).

- **UI:** Tauri ou Electron, ou Qt se quiser nativo
- **Render:** FFmpeg para I/O e filtros, shaders para efeitos em tempo real
- **IA:** ONNX Runtime com execução local — sem cota, sem créditos
- **Material:** descritores JSON locais, sem dependência de CDN

Próximo passo recomendado

Começar por **remover fundo + legendas automáticas**. São os dois recursos que mais doem no paywall, têm as melhores implementações abertas e entregam valor visível num MVP pequeno.